

P1960R0

NB Comment Changes Reviewed by SG1

Published Proposal, 2019-11-07

This version:

<https://wg21.link/P1960R0>

Author:

[David Olsen](#) (NVIDIA)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Abstract

2 US355

2.1 Poll

2.2 Wording

3 US358

3.1 Poll

3.2 Wording

4 US359

4.1 Poll

4.2 Wording

5 US356

5.1 Polls

5.2 Wording

6 US364

6.1 Poll

6.2 Wording

§ 1. Abstract

All of the changes that resulted from NB comments reviewed by SG1 in Belfast that are not already covered by existing papers.

§ 2. US355

§ 2.1. Poll

Recommend to make `notify_one`, `notify_all`, and `operator=(float)` of `atomic_ref` `const`, in response to US 358 and [US 355](#): Unanimous consent

§ 2.2. Wording

Change all the declarations of member functions `notify_one` and `notify_all` in class `atomic_ref` to be `const`.

In 31.7 "Class template `atomic_ref`" [**atomics.ref.generic**] near the end of the class synopsis:

```
void notify_one() const noexcept;  
void notify_all() const noexcept;
```

In 31.7.1 "Operations" [**atomics.ref.ops**] near the end of the list of functions:

```
void notify_one() const noexcept;  
  
[ ... skip ... ]  
  
void notify_one() const noexcept;
```

In 31.7.2 "Specializations for integral types" [**atomics.ref.int**] near the end of the class synopsis:

```
void notify_one() const noexcept;  
void notify_all() const noexcept;
```

In 31.7.3 "Specializations for floating-point types" [**atomics.ref.float**] near the end of the class synopsis:

```
void notify_one() const noexcept;  
void notify_all() const noexcept;
```

In 31.7.4 "Partial specializations for pointers" [**atomics.ref.pointer**] near the end of the class synopsis:

```
void notify_one() const noexcept;  
void notify_all() const noexcept;
```

§ 3. US358

§ 3.1. Poll

Recommend to make `notify_one`, `notify_all`, and `operator=(float)` of `atomic_ref` `const`, in response to [US 358](#) and US 355: Unanimous consent

§ 3.2. Wording

In 31.7.3 "Specializations for floating-point types" [**atomics.ref.float**] near the middle of the class synopsis:

```
floating-point operator=(floating-point) const noexcept;
```

§ 4. US359

§ 4.1. Poll

Recommend adopting the comment's proposed wording (i.e. ignoring the alternative), in response to [US 359](#):

SF F N A SA

5 12 1 0 0

Unanimous consent

LWG recommended using `value_type` as the return type, and making the same change to the identical wording in `atomic`.

§ 4.2. Wording

Change 31.7.5 "Member operators common to integers and pointers to objects" [**atomics.ref.memop**] as follows:

```
T* value_type operator++(int) const noexcept;
```

Effects: Equivalent to `return fetch_add(1);`

```
T* value_type operator--(int) const noexcept;
```

Effects: Equivalent to `return fetch_sub(1);`

```
T* value_type operator++() const noexcept;
```

Effects: Equivalent to `return fetch_add(1) + 1;`

```
T* value_type operator--(int) const noexcept;
```

Effects: Equivalent to `return fetch_sub(1) - 1;`

Change 31.8.5 "Member operators common to integers and pointers to objects" [**atomics.types.memop**] as follows:

⚠ `value_type` `operator++(int)` `volatile noexcept`;

⚠ `value_type` `operator++(int)` `noexcept`;

Effects: Equivalent to `return fetch_add(1);`

⚠ `value_type` `operator--(int)` `volatile noexcept`;

⚠ `value_type` `operator--(int)` `noexcept`;

Effects: Equivalent to `return fetch_sub(1);`

⚠ `value_type` `operator++()` `volatile noexcept`;

⚠ `value_type` `operator++()` `noexcept`;

Effects: Equivalent to `return fetch_add(1) + 1;`

⚠ `value_type` `operator--()` `volatile noexcept`;

⚠ `value_type` `operator--()` `noexcept`;

Effects: Equivalent to `return fetch_sub(1) - 1;`

§ 5. US356

§ 5.1. Polls

Recommend removing the instance-dependence of `atomic_ref::is_lock_free()`, in response to [US 356](#):

SF F N A SA

3 3 5 1 0

Consensus for change.

Recommend adding an instance-dependent `atomic_ref::is_instance_lock_free()`, in response to US 356:

SF F N A SA

2 5 3 1 2

Not consensus for change in an NB ballot process

Recommend making `atomic_ref::is_lock_free()` static, in response to US 356:

SF F N A SA

3 5 3 1 2

Not consensus for change in an NB ballot process

§ 5.2. Wording

Change the specification of `atomic_ref<T>::is_lock_free` in 31.7.1 "Operations" [atomics.ref.ops] p4:

```
bool is_lock_free() const noexcept;
```

Returns: true if ~~the object's~~ operations on all objects of the type `atomic_ref<T>` are lock-free, false otherwise.

§ 6. US364

§ 6.1. Poll

No object to unanimous consent to adopting the wording in this paper as the resolution to [US364](#).

§ 6.2. Wording

Change the specification of `counting_semaphore<N>::try_acquire` in 32.7.2 "Class template `counting_semaphore`" [thread.sema.cnt] as follows:

```
bool try_acquire() noexcept;
```

~~Effects:~~

- ~~- With low probability, returns immediately. An implementation should ensure that `try_acquire` does not consistently return false in the absence of contending acquisitions.~~
- ~~- Otherwise, atomically check whether counter is greater than zero and, if so, decrement counter by one.~~

Effects: Attempts to atomically decrement counter if it is positive, without blocking. If counter is not decremented, there is no effect and `try_acquire` immediately returns. An implementation may fail to decrement counter even if it is positive. [Note: This spurious failure is normally uncommon, but allows interesting implementations based on a simple compare and exchange ([atomic]). -- end note] An implementation should ensure that `try_acquire` does not consistently return false in the absence of contending semaphore operations.

Returns: true if counter was decremented, otherwise false.

